

Cahier des Charges - API Simple Blog

1. Contexte et Objectif

L'application vise à offrir une API simple permettant de créer, lire, modifier et supprimer des articles de blog. Elle sert de support pour apprendre :

- Le développement d'une API REST en Node.js
 - La gestion d'une base de données MongoDB (via Mongoose)
 - La structuration d'un projet backend
 - Les bonnes pratiques d'organisation de code
-

2. Technologies

Élément	Technologie
Langage	JavaScript
Runtime	Node.js
Framework serveur	Express.js
Base de données	MongoDB
ODM - Object Data Modeling *	Mongoose
Outils	Nodemon, dotenv, Postman

3. Fonctionnalités à implémenter

3.1. Routes

3.1.1. Auth

- **POST** `/auth/register`
 - Crée un nouvel utilisateur (hash du mot de passe)
 - En réponse : statut 201 + message ou données utilisateur (sans le mot de passe)
- **POST** `/auth/login`
 - Vérifie email et mot de passe
 - Si OK : génère un JWT (ex. payload `{ id, username, role }`)
 - En réponse : statut 200 + `{ token: "Bearer ..." }`
- **Middleware** `verifyToken`
 - Vérifie la présence et la validité du token dans l'en-tête `Authorization`
 - Injecte `req.user = { id, username, role }`

3.1.2. Articles

Verbe	Route	Protection	Description
GET	<code>/posts</code>	publique	Récupère tous les articles (avec tri/filtre)
GET	<code>/posts/:id</code>	publique	Récupère un article par ID
POST	<code>/posts</code>	<code>verifyToken</code>	Crée un nouvel article (req.user comme author)
PUT	<code>/posts/:id</code>	<code>verifyToken</code>	Met à jour (seul l'auteur ou admin)
DELETE	<code>/posts/:id</code>	<code>verifyToken</code>	Supprime (seul l'auteur ou admin)

(routes protégées sauf récupération publique)

3.1.3. Query params pour GET `/posts`

- `?sort=desc` ou `?sort=asc`
 - Trie par `createdAt` (descendant par défaut si non précisé)
- `?category=<nom>`
 - Filtre les articles dont `category === nom`
- Combinaisons possibles : `/posts?category=Tech&sort=desc`

3.1.4. Gestion des erreurs & validations

- 400 si `title`, `content`, ou champs d'inscription invalides
- 401 si JWT manquant ou invalide
- 403 si tentative d'édition/suppression par un non-auteur (et non admin)
- 404 si ressource introuvable
- 500 pour erreur serveur

Structure d'un article :

```
{
  title:      String,          // obligatoire
  content:    String,          // obligatoire
  author:     mongoose.ObjectId, // référence vers User
  category:   String,          // ex. "Tech", "Sport", etc.
  createdAt:  Date,            // automatique
  updatedAt:  Date            // automatique
}
```

Structure d'un utilisateur :

```
{
  username: String,          // obligatoire
  email:     String,          // unique, obligatoire
  password:  String,          // hashé avec bcrypt,
obligatoire
  role:      String          // ex. "user" ou "admin"
}
```

3.2. Validation des données

- Vérifier que `title` et `content` ne sont pas vides lors de la création ou modification d'un article
-

4. Contraintes pédagogiques

- Code clair, commenté, structuré
 - Utilisation d' `async/await`
 - Pas de base de données relationnelle
-

5. Structure recommandée du projet

```
/blog-api
|
├── controllers/
|   ├── authController.js
|   └── postController.js
|
├── middlewares/
|   ├── verifyToken.js
|   └── isAdmin.js
|
├── models/
|   ├── User.js
|   └── Post.js
|
├── routes/
|   ├── authRoutes.js
|   └── postRoutes.js
|
├── utils/
|   └── errorHandler.js
|
├── app.js
├── .env
└── .gitignore
```

```
|— package.json
|— README.md
```

6. Livrables attendus

- **Code source** sur GitHub
- **README.md** mis à jour avec :
 - **Instructions d'installation**
 - **Liste des routes (auth + posts)**
 - **Exemples de requêtes** (Insomnia/Thunder) pour :
 - Inscription, connexion
 - Création, modification, suppression protégées
 - Tri et filtrage

→ *ODM* : Techniques de programmation offrant une abstraction de haut niveau pour les interactions avec les bases de données. Elles permettent d'interagir avec les bases de données grâce à des concepts de programmation orientée objet, rendant les opérations de base de données plus automatiques et moins dépendantes des requêtes brutes.*